

Bessel Spur Removal Algorithm

Fixed-Point Implementation Specification

Document ID	LLR_ALG_002
Version	1.0
Status	Released
Parent HLR	HLR_003 — RF Signal Processing
Depends on	LLR_ALG_000, LLR_ALG_001, LLR_ALG_006
Pipeline stage	Stage 1 of 4

Scope. This document specifies the complete behavioral, mathematical, and fixed-point arithmetic requirements for the Bessel notch spur removal stage of the RF processing pipeline. It contains no implementation code. All parameters are derived from LLR_ALG_000 (Arithmetic Target Specification). The specification is sufficient for a code generator or a human implementer to produce a correct, verifiable implementation on any 32-bit fixed-point target.

Contents

1	Purpose and Context	2
1.1	Role in the pipeline	2
1.2	Why Bessel filters	2
2	Arithmetic Target	2
3	Signal Specification	3
3.1	Input signal	3
3.2	Output signal	3
4	Spur Detection Specification	3
4.1	Definition of a spur	3
4.2	Detection method	3
4.3	Detection algorithm	3
5	Filter Design Specification	4
5.1	Biquad transfer function	4
5.2	Bessel notch coefficient design	4
5.3	Coefficient conversion to Q4.27	4
5.4	Coefficient specification	5
6	Filter Topology and State	5
6.1	Structure: Direct Form II Transposed	5
6.2	Filter state specification	6
6.3	Cascade specification	6
7	Fixed-Point Operation Groups	6
7.1	Group 1 — Coefficient design (initialization only)	6
7.2	Group 2 — Biquad output $y[n]$ (inner loop, per sample)	7
7.3	Group 3 — State update $w_1[n]$ (inner loop, per sample)	7
7.4	Group 4 — State update $w_2[n]$ (inner loop, per sample)	7
7.5	Group 5 — Cascade (N biquads in sequence)	8
8	Saturation and Overflow Budget	8
8.1	Saturation rule	8
8.2	Overflow budget	8
9	Configuration Interface	8
10	Processing Interface	9
10.1	process() function	9
10.2	reset() function	9
11	Constraints	9
12	Initialization Sequence	10
13	Verification Requirements	10

1 Purpose and Context

1.1 Role in the pipeline

The Bessel spur removal stage removes narrowband parasitic signals (spurs) from IQ samples before phase computation. It operates on the output of the ADC promotion stage (LLR_ALG_001) and feeds the instantaneous phase computation stage (LLR_ALG_003).

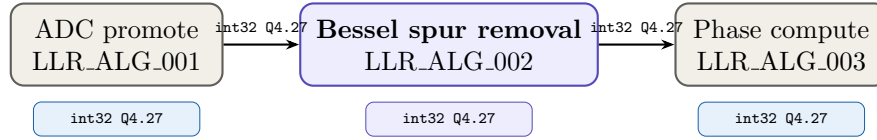


Figure 1: Position of the Bessel spur removal stage in the RF pipeline. Format is preserved: input and output are both `int32 Q4.27`.

1.2 Why Bessel filters

Filter type	Group delay	Phase distortion	IQ suitability
Butterworth	non-linear	yes	moderate
Chebyshev I	non-linear	significant	poor
Elliptic	non-linear	significant	poor
Bessel	maximally flat	none	best

Table 1: Filter type comparison for IQ signal processing.

Bessel filters are the only filter family with maximally flat group delay, meaning all frequency components of the signal experience the same propagation delay. This is essential for IQ processing: any differential delay between I and Q channels degrades the phase measurement after demodulation. The softer frequency rolloff of Bessel filters is acceptable given that spur removal is the only objective of this stage.

2 Arithmetic Target

All fixed-point parameters in this document are derived from LLR_ALG_000. No parameter is defined independently here.

Parameter	Value	Derivation	Source
Word size	32 bits	CPU register width	LLR_ALG_000 §3.1
Integer bits	4	signal range analysis	LLR_ALG_000 §3.2
Fractional bits	27	$32 - 1 - 4 = 27$	LLR_ALG_000 §3.3
Q format	Q4.27	derived	LLR_ALG_000 §3.4
Accumulator width	64 bits	2×32	LLR_ALG_000 §4.2
Multiply shift	$\gg 27$	= fractional bits	LLR_ALG_000 §4.1
Multiply format	Q8.54	$Q4.27 \times Q4.27$	LLR_ALG_000 §4.1
Saturation width	32 bits	= word size	LLR_ALG_000 §4.1

Table 2: Fixed-point parameters derived from LLR_ALG_000.

3 Signal Specification

3.1 Input signal

Property	Specification	Notes
Channel layout	Two independent channels	I and Q processed separately
Type	<code>int32_t</code> array	
Fixed-point format	Q4.27	from LLR_ALG_000
Value range	$[-\pi, +\pi]$ nominal	IQ amplitude bounded
Samples per buffer	N (configurable, ≤ 256)	
Source	LLR_ALG_001 output	ADC promote stage

Table 3: Input signal specification.

3.2 Output signal

Property	Specification	Notes
Channel layout	Two independent channels	same structure as input
Type	<code>int32_t</code> array	
Fixed-point format	Q4.27	format preserved
Processing	In-place	output overwrites input buffer
Destination	LLR_ALG_003 input	phase computation stage

Table 4: Output signal specification.

4 Spur Detection Specification

4.1 Definition of a spur

Definition. A spur is a narrowband spectral component whose power exceeds the local noise floor by a configurable threshold T_{dB} . Spurs arise from harmonic distortion, local oscillator leakage, clock coupling, or other periodic interference sources in the RF front-end.

4.2 Detection method

Property	Specification	Notes
Trigger	Once per buffer, before filtering	not per sample
Domain	Frequency domain	spectral analysis
Arithmetic	Float permitted in this step only	detection, not inner loop
Threshold	Configurable, expressed in dB	above local noise floor
Output	List of spur frequencies $f_{s,k}$ in Hz	up to K_{max} entries
Max simultaneous spurs	$K_{max} \leq 4$ (static allocation)	configurable

Table 5: Spur detection specification.

4.3 Detection algorithm

1. Compute the magnitude spectrum of the input buffer.

2. Estimate the local noise floor across the spectrum.
3. Identify spectral peaks satisfying:

$$P(f_k) > P_{\text{noise}}(f_k) + T_{dB} \quad (1)$$

where $P(f_k)$ is the power at frequency f_k in dB, $P_{\text{noise}}(f_k)$ is the local noise floor estimate at f_k , and T_{dB} is the configurable detection threshold.

4. Record the frequencies of detected peaks as $\{f_{s,1}, f_{s,2}, \dots, f_{s,K}\}$ with $K \leq K_{max}$.
5. If $K > K_{max}$, retain the K_{max} strongest peaks.

5 Filter Design Specification

5.1 Biquad transfer function

Each detected spur at frequency $f_{s,k}$ is attenuated by one second-order IIR notch section (biquad). The transfer function is:

$$H_k(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}} \quad (2)$$

5.2 Bessel notch coefficient design

Coefficients are designed in floating-point at initialization time. Float is permitted in this step only. The design procedure for a notch at normalized frequency $\omega_0 = 2\pi f_{s,k}/f_s$ is:

$$\alpha = \sin(\omega_0) \sinh\left(\frac{\ln 2}{2} \cdot \frac{\Delta\omega}{\omega_0}\right) \cdot \frac{\omega_0}{\sin(\omega_0)} \quad (3)$$

$$b_0 = \frac{1}{1 + \alpha} \quad (4)$$

$$b_1 = \frac{-2 \cos(\omega_0)}{1 + \alpha} \quad (5)$$

$$b_2 = \frac{1}{1 + \alpha} \quad (6)$$

$$a_1 = \frac{-2 \cos(\omega_0)}{1 + \alpha} \quad (7)$$

$$a_2 = \frac{1 - \alpha}{1 + \alpha} \quad (8)$$

where $\Delta\omega$ is the configurable notch bandwidth (default: $\omega_0/10$) and f_s is the ADC sample rate in Hz.

5.3 Coefficient conversion to Q4.27

After floating-point design, each coefficient is converted to Q4.27:

$$c_{Q4.27} = \text{round}(c_{\text{float}} \times 2^{27}) \quad (9)$$

Rule. Coefficient conversion uses floating-point arithmetic and is performed once at initialization, before the first buffer is processed. It must not occur inside the processing loop. Once converted, coefficients are stored as `int32_t Q4.27` and are not recomputed unless the spur frequency changes.

5.4 Coefficient specification

Coefficient	Role	Format	Typical range
b_0	numerator coefficient	int32 Q4.27	[0.5, 1.0]
b_1	numerator coefficient	int32 Q4.27	[-2.0, +2.0]
b_2	numerator coefficient	int32 Q4.27	[0.5, 1.0]
a_1	denominator coefficient	int32 Q4.27	[-2.0, +2.0]
a_2	denominator coefficient	int32 Q4.27	[-1.0, +1.0]

Table 6: Biquad coefficient specification. $a_0 = 1$ is implicit (normalized).

6 Filter Topology and State

6.1 Structure: Direct Form II Transposed

Definition. The Direct Form II Transposed (DFII-T) structure minimizes the number of delay elements (2 per biquad) and exhibits better numerical behavior than Direct Form I for fixed-point implementations, because intermediate values remain bounded by the coefficient magnitudes rather than growing with input amplitude.

The signal flow for one biquad is:

$$y[n] = b_0 \cdot x[n] + w_1[n-1] \quad (10)$$

$$w_1[n] = b_1 \cdot x[n] - a_1 \cdot y[n] + w_2[n-1] \quad (11)$$

$$w_2[n] = b_2 \cdot x[n] - a_2 \cdot y[n] \quad (12)$$

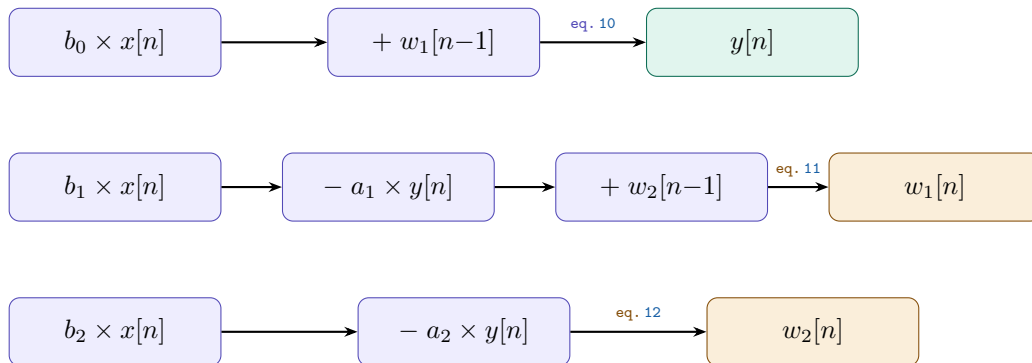


Figure 2: Direct Form II Transposed signal flow for one biquad section. Purple: arithmetic operations. Amber: state variables.

6.2 Filter state specification

Variable	Type	Format	Initial value	Persists across buffers
w_1	int64	Q8.54	0	yes
w_2	int64	Q8.54	0	yes

Table 7: Filter state variable specification. State is int64 Q8.54 because it stores products before renormalization.

Rule. State variables w_1 and w_2 are stored as `int64` in Q8.54 format — they hold products before the renormalization shift. Truncating them to `int32` between samples would lose $frac_bits = 27$ bits of precision and introduce systematic error proportional to the signal amplitude. State variables persist across DMA buffer boundaries and must not be reset between consecutive calls to the processing function.

6.3 Cascade specification

Property	Specification
Number of stages	One biquad per detected spur ($K \leq K_{max}$)
Stage connection	Output of stage k is input of stage $k + 1$
Format at stage boundary	<code>int32</code> Q4.27 (each stage saturates its own output)
Independent per channel	I and Q channels processed with separate state machines
Max stages	$K_{max} = 4$ (static allocation, no dynamic memory)

Table 8: Cascade topology specification.

7 Fixed-Point Operation Groups

This section specifies the complete fixed-point arithmetic behavior of the inner processing loop. Each row corresponds to exactly one elementary operation from the fixed-point taxonomy (FIX-POINT_TAXONOMY.md). No implementation code appears in this section.

Column definitions. **FP IN:** type and Q format of the input value. **Operation:** one elementary operation (arithmetic, shift, or saturate). **FP OUT:** type and Q format of the output value. **Shift:** shift amount if this step is a format-shaping shift. **Sat:** whether saturation to `word_size` bits is applied at this step.

7.1 Group 1 — Coefficient design (initialization only)

Float is permitted in this group. It does not execute in the inner loop.

Step	FP IN	Operation	FP OUT	Shift	Sat
1.1	float	Bessel notch design (eq. 3–8)	float	none	no
1.2	float	multiply by 2^{27}	float	none	no
1.3	float	round to nearest integer (eq. 9)	int32 Q4.27	none	no

Table 9: Group 1: coefficient design at initialization. Float permitted.

7.2 Group 2 — Biquad output $y[n]$ (inner loop, per sample)

Implements equation 10: $y[n] = b_0 \cdot x[n] + w_1[n - 1]$.

Step	FP IN	Operation	FP OUT	Shift	Sat
2.1	int32 Q4.27 (b_0)	widen to int64	int64 Q4.27	none	no
2.2	int32 Q4.27 ($x[n]$)	widen to int64	int64 Q4.27	none	no
2.3	int64 Q4.27 $\times$ int64 Q4.27	multiply	int64 Q8.54	none	no
2.4	int64 Q8.54 + int64 Q8.54 (w_1)	add state w_1	int64 Q8.54	none	no
2.5	int64 Q8.54	shift right (renormalize)	int64 Q4.27	$\gg 27$	no
2.6	int64 Q4.27	saturate to 32 bits	int32 Q4.27	none	yes/32

Table 10: Group 2: output sample computation. Step 2.6 is the only saturation in the inner loop. $y[n]$ = output of step 2.6. Format preserved: output is **int32 Q4.27**.

7.3 Group 3 — State update $w_1[n]$ (inner loop, per sample)

Implements equation 11: $w_1[n] = b_1 \cdot x[n] - a_1 \cdot y[n] + w_2[n - 1]$.

Step	FP IN	Operation	FP OUT	Shift	Sat
3.1	int32 Q4.27 (b_1), int32 Q4.27 ($x[n]$)	widen + multiply	int64 Q8.54	none	no
3.2	int32 Q4.27 (a_1), int32 Q4.27 ($y[n]$)	widen + multiply	int64 Q8.54	none	no
3.3	int64 Q8.54 – int64 Q8.54	subtract	int64 Q8.54	none	no
3.4	int64 Q8.54 + int64 Q8.54 (w_2)	add state w_2	int64 Q8.54	none	no
3.5	int64 Q8.54	store as w_1	int64 Q8.54	none	no

Table 11: Group 3: state w_1 update. No renormalization or saturation: w_1 is stored as **int64 Q8.54**. Renormalization occurs in Group 2 step 2.5 when w_1 is consumed.

7.4 Group 4 — State update $w_2[n]$ (inner loop, per sample)

Implements equation 12: $w_2[n] = b_2 \cdot x[n] - a_2 \cdot y[n]$.

Step	FP IN	Operation	FP OUT	Shift	Sat
4.1	int32 Q4.27 (b_2), int32 Q4.27 ($x[n]$)	widen + multiply	int64 Q8.54	none	no
4.2	int32 Q4.27 (a_2), int32 Q4.27 ($y[n]$)	widen + multiply	int64 Q8.54	none	no
4.3	int64 Q8.54 – int64 Q8.54	subtract	int64 Q8.54	none	no
4.4	int64 Q8.54	store as w_2	int64 Q8.54	none	no

Table 12: Group 4: state w_2 update. No renormalization or saturation: w_2 is stored as **int64 Q8.54**.

7.5 Group 5 — Cascade (N biquads in sequence)

Step	FP IN	Operation	FP OUT	Shift	Sat
5.1	int32 Q4.27	input to biquad k (Groups 2, 3, 4)	int32 Q4.27	—	per stage
5.2	int32 Q4.27	output of biquad k = input of biquad $k + 1$	int32 Q4.27	none	no

Table 13: Group 5: cascade connection. Format `int32 Q4.27` is preserved at every stage boundary. Each stage applies Group 2 step 2.6 saturation independently.

8 Saturation and Overflow Budget

8.1 Saturation rule

Saturation rule. Saturation to `word_size` = 32 bits is applied *exactly once* per biquad section, at Group 2 step 2.6, when the renormalized `int64 Q4.27` value is stored to the `int32 Q4.27` output buffer. Saturation is *never* applied to state variables w_1 and w_2 between samples: they accumulate in `int64` without intermediate clamping.

8.2 Overflow budget

The Q4.27 format supports values in the range $[-2^4, +2^4) = [-16, +16)$.

The cascade gain G_k of biquad k is bounded by:

$$G_k \leq |b_0| + |b_1| + |b_2| \quad (13)$$

The total cascade gain for K stages must satisfy:

$$G_{\text{total}} = \prod_{k=1}^K G_k < 2^{\text{int_bits}} = 16 \quad (14)$$

In practice, a 50% safety margin is applied:

$$G_{\text{total}} < 8.0 \quad (\text{practical constraint}) \quad (15)$$

Rule. Equation 15 must be verified at initialization using floating-point arithmetic, before the first buffer is processed. It must not be checked at runtime in the processing loop. If the cascade gain exceeds 8.0, the coefficient design must be revised (reduce notch bandwidth or reduce number of stages).

9 Configuration Interface

Parameter	Type	Unit	Default	Description
f_s	float	Hz	—	ADC sample rate
T_{dB}	float	dB	20.0	Spur detection threshold above noise floor
K_{max}	size_t	—	4	Maximum simultaneous notch filters
$\Delta\omega$	float	rad	$\omega_0/10$	Notch bandwidth per spur

Table 14: Configuration parameters. All set at initialization. None may change during processing without a full reset.

10 Processing Interface

10.1 process() function

Parameter	Direction	Type	Format	Description
iBuffer	in/out	int32_t array	Q4.27	I channel, N samples (in-place)
qBuffer	in/out	int32_t array	Q4.27	Q channel, N samples (in-place)
len	in	size_t	—	Number of samples (≤ 256)
return	out	int32_t	—	Number of spurs removed, or < 0 on error

Table 15: process() interface specification.

10.2 reset() function

Resets all state variables to zero. Must be called on stream restart. Must not be called between consecutive normal buffer calls.

State cleared	Type	Value after reset
w_1 (per biquad, per channel)	int64 Q8.54	0
w_2 (per biquad, per channel)	int64 Q8.54	0
Spur frequency table	float array	0.0
Active spur count	size_t	0

Table 16: State cleared by reset().

11 Constraints

1. **No float in the inner loop.** Float arithmetic is permitted only in Group 1 (initialization). All operations in Groups 2, 3, 4, and 5 use integer arithmetic only.
2. **No dynamic memory allocation.** The filter bank (coefficients and state) is statically allocated. Size is fixed at compile time by K_{max} .
3. **Both operands widened before multiply.** Both operands of every multiplication must be widened to 64-bit before the operation. Multiplying two `int32` values directly is forbidden.
4. **Saturation at every int32 store.** Every renormalized value written to an `int32` output must pass through saturation. No unsaturated `int32` store.
5. **State variables are int64.** w_1 and w_2 must remain `int64` between samples. They must never be truncated to `int32` mid-accumulation.
6. **State persists across buffer boundaries.** The filter state must not be reset between consecutive `process()` calls. Resetting between buffers introduces discontinuities.
7. **Cascade gain verified at initialization.** Equation 15 ($G_{total} < 8.0$) must be verified once at startup, not at runtime.

8. **I and Q channels are independent.** The I and Q channels are processed with separate state machines. They must not share state variables.

12 Initialization Sequence

Step	Action	Arithmetic	Verification
1	Verify $\text{frac_bits} + \text{int_bits} + 1 == 32$	—	assert
2	Run spur detection on first buffer	float allowed	$K \leq K_{max}$
3	Design Bessel notch coefficients (Group 1)	float	$ c < 2.0$ per coeff
4	Convert coefficients to Q4.27 (eq. 9)	float then int32	range check
5	Verify cascade gain $G_{total} < 8.0$	float	abort if exceeded
6	Initialize all state variables to zero	—	$w_1 = w_2 = 0$

Table 17: Initialization sequence. Steps 1–6 run once before first `process()` call.

13 Verification Requirements

ID	Description	FP IN	Expected FP OUT
T01	Clean sine, no spur	int32 Q4.27	output $\approx$ input, error < 1 LSB
T02	Sine + spur at f_0	int32 Q4.27	spur attenuated > 40 dB at f_0
T03	Two spurs at f_0, f_1	int32 Q4.27	both attenuated > 40 dB
T04	$K = K_{max} = 4$ spurs	int32 Q4.27	all 4 attenuated > 40 dB
T05	State continuity, 2 consecutive buffer calls	int32 Q4.27	no discontinuity at boundary
T06	$K_{max} = 0$ (no-op)	int32 Q4.27	output = input exactly
T07	Cascade gain verification	float coeffs	assert $G_{total} < 8.0$
T08	State variables remain int64	—	no int32 truncation between samples
T09	Saturation: INT32_MAX input	int32 Q4.27	no overflow at output
T10	Coefficient conversion accuracy	float	int32 Q4.27, error < 1 LSB
T11	<code>reset()</code> clears all state	int64 Q8.54	$w_1 = w_2 = 0$ after reset
T12	I and Q independent	int32 Q4.27	different state machines, no cross-coupling